If the graceful shutdown does not work, the next option is to terminate the query which sends a SIGTERM signal. These are server signaling System Administration Functions.[8] Terminating a running query may cause data loss.

Use the following function to terminate the backend. Multiple queries in progress may be associated with the backend process so only use this option when it's really needed.

```
SELECT PG_TERMINATE_BACKEND(pid);
```

Later you'll learn about safeguards you can put in place to help reduce the likelihood of getting into a state where you need to terminate a query.

Besides current query activities, sometimes queries can get blocked on each other. One of the key design features of PostgreSQL is how it locks and unlocks resources to handle a lot of concurrent writing and reading activity.

In the next section you'll peek into some of the details on lock activity.

Glancing At Current Lock Behavior

PostgreSQL uses pessimistic locking, which means lockable resources are locked up front before they're modified. When queries try to access locked resources they can either share access or they are blocked and must wait to acquire a lock that is held from a single statement. Lockable resources include tables and also rows within tables.

Although a deep dive into locks is beyond the scope of this chapter, knowing some of the basic lock types can help you design solutions that minimize lock contention.

Exclusive locks should be minimized because they block other queries trying to access the same resources. The length of the time the lock is taken can grow with the row count of the table. For example when a table modification is happening that locks the table, the lock duration will be longer for a table with more rows because each row needs to change. If the same table is queried in high volume during a long lock duration period, it is very likely those queries will be blocked and result in being cancelled and causing application errors.

Although Locks can be created explicitly, usually resources are locked from implicit locks taken by various SQL statements. PostgreSQL documentation

8. https://www.postgresql.org/docs/current/functions-admin.html